Class 6: R functions

Qiman Hu A17912409

Table of contents

Background

All functions in R have at least 3 things:

- a *name* (we pick and use it to call the function)
- input *arguments* (one or more comma seperated inputs that go inside the brackets when we call the function)
- the *body* (the lines of R code that do the work of the function)

A first function

Here we will create a function to add some numbers. Let's call it `add()`.

Input arguments can be either **"required"** or **"optional"**. The later will have fall-back default values that will be used if the user does not specify them.

```
add <- function(x,y=0) {
  x + y

}
```

Can we use our new function:

```
add(10,100)
```

```
[1] 110
```

```
add(10)
```

```
[1] 10
```

Version 2: handle either two inputs OR a vector

```
add <- function(x,y=0) {
 sum(x,y)

}
```

```
add(4,7)
```

```
[1] 11
```

```
add(c(4,7,10))
```

```
[1] 21
```

Version 3: add any number of inputs that the user provides >Q1c. create a third version of your function that can add any number of inputs that the user provides. For example, add(1, 2, 3, -4) should return 2.

```
add <- function(...) {
 sum(...)

}
```

```
add(1,2,3,-4)
```

```
[1] 2
```

We can explicitily set a **return** value output from a function (rather than the default last line) by usng the `return()` function call.

```
add <- function(x,y=0, z=0) {
 return(sum(x,y,z))
  cat("Is it break time yet?/n")
}

add(10,100)
```

```
[1] 110
```

Write a generate_dna() function

A useful funciton here is the "base R" `sample()` function:

```
sample(1:5, size=10, replace=TRUE)
```

```
 [1] 4 5 4 1 3 1 3 2 5 5
```

We can use this to make a random nucleotide sequence if we draw from "A", "C", "T", "G".

```
sample(x=c("A", "C", "T", "G"), size=10, replace=TRUE)
```

```
 [1] "A" "T" "T" "A" "T" "G" "A" "G" "A" "T"
```

> Q2a. Write a function `generate_dna()` that returns a random DNA sequence of a length specified by the user. a. Your first version should return a multi-element vector of single character nucleotides. For example generate_dna(6) might return "A", "T", "T", "G", "A", "C". [1 pt]

```
generate_dna<- function(len){
 bases <- c("A", "C", "T", "G")

 dnaseq <-sample(bases, size=len, replace=TRUE)

  return(dnaseq)
}
```

```
generate_dna(20)
```

```
 [1] "C" "A" "C" "A" "G" "T" "C" "A" "G" "A" "T" "G" "T" "C" "T" "G" "A" "A" "C"
[20] "C"
```

Q2b. Your second version should optionally be able to return either a multi-element vector of single character nucleotides (as before) or a **single character string** (not a vector of single letters but a singe vector of multiple letters). For example "AAGGCTG". [1 pt]

Functions that could be useful here are `paste()`, `if()`, `cat()`, and `return()`

```
generate_dna <- function(n, as_string = FALSE) {
  dna <- sample(c("A","T","G","C"), n, TRUE)
  if (as_string) paste(dna, collapse="") else dna
}
```

```
generate_dna(8)
```

```
[1] "C" "A" "A" "A" "G" "A" "A" "C"
```

```
generate_dna(8, TRUE)
```

```
[1] "CGACTCTG"
```

Q2c. Finally, create a final version of your function that prints out a FASTA format sequence with an id line indicating the sequence length.

```
#This function generates a random DNA sequence of length n and prints it in FASTA format.

generate_dna <- function(n){
  #created a vector of possible DNA bases
  bases <- c("A","T","G","C")

  #randomly choose n bases, allowing repeats
  dnaseq <- sample(bases,n,replace=TRUE)

  #joining the bases into one DNA string

  dna_string <- paste(dnaseq, collapse="")

  #print the FASTA header and the DNA sequence
  cat(">len", n, "\n", sep="")
```

```
  cat(dna_string, "\n", sep="")

  #return the DNA sequence string

  return(dna_string)
}
```

```
generate_dna(10)
```

```
>len10
GATGGACATT

[1] "GATGGACATT"
```

Write a generate_protein() function

> Q3. Write a function generate_protein() that returns a random peptide/protein sequence of a length specified by the user. For example generate_protein(6) might return "WQRTAG".

```
#this function generates a random protein sequence of length n using the single letter codes

generate_protein <- function(n) {

  #created a vector containing the 20 standard amino acid letters

  aa <- c("A", "R", "N", "D", "C",
          "Q", "E", "G", "H", "I",
         "L", "K", "M", "F", "P",
          "S", "T", "W", "Y", "V")
  #randomely choose n amino acids

  protein <- sample(aa,n,replace=TRUE)

  #join the letters together in one single string and return the final protein sequence
  protein_string <- paste(protein, collapse="")

  return(protein_string)

}
```

```
generate_protein(10)
```

```
[1] "LAAHVSRHES"
```

Generate random protein sequences of length 6 to 13

> Q4. Adapt and use your generate_protein() function to generate a series of random protein sequences ranging from 6 to 13 amino acids in length (one sequence per length). Take advantage of the base R function for() or sapply() so that you do not have to call generate_protein() eight times by hand. Be sure to format your results in FASTA format ready to paste or upload as a query to NCBI BLASTp.

```
# function to generate random protein sequence
generate_protein <- function(n) {
  aa <- c("A","R","N","D","C","Q","E","G","H","I",
          "L","K","M","F","P","S","T","W","Y","V")
  paste(sample(aa, n, replace = TRUE), collapse = "")
}

# lengths 6 to 13
leng <- 6:13

# use sapply and print FASTA format
invisible(
  sapply(leng, function(i) {
    cat(paste0(">id.", i, "\n", generate_protein(i), "\n"))
  })
)
```

```
>id.6
MPNVRV
>id.7
PRSVSNK
>id.8
LVNHYNIL
>id.9
TTMFAEINQ
>id.10
VQESSYGNPF
>id.11
```

```
ECNLWWKNYMD
>id.12
EEHRREATYACK
>id.13
PNQRCWSLAMCIE
```

BLASTp search against nr- are our peptides "unique in nature?"

> Q5:Take your FASTA-formatted peptides from Q4 and run them as a single BLASTp search against the Non-redundant protein sequences (nr) database at https://blast.ncbi.nlm.nih.gov/. For this question do not restrict the organism (leave the Organism field blank so that the full nr database is searched).

Length(aa)	Best hit %identity	Best hit % coverage	Unique? (Y/N)
6	100.00	100	N
7	100.00	100	N
8	87.50	100	Y
9	100.00	89	Y
10	100	80	Y
11	90.00	91	Y
12	81.82	92	Y
13	75.00	69	Y

6a. At sequence length 8 the randomly generate peptides started to look "unique in nature

6b.Short random peptides are almost always found in nr because their sequence space (20^L) is relatively small and heavily sampled by the vast number of already known protein sequences. Longer peptides occupy an exponentially larger sequence space. This size exceeds the size of the known protein universe making 100% matches much less likely.

Connecting your finding to immunology

> Q6.In 3–6 sentences total and using your Q5 data and the reasoning from Q5b, what do you think this minimum length is and why might it be a bad design choice for the immune system to present very short peptides? [3 pt]

Based on the results from question 5, the minimum peptide length where sequences begin to appear unique is around 8 amino acids. Shorter peptides match many sequences in nr while longer ones showed matches of lower %identity and %coverage. For MHC class II molecules it might be a bad design choice because short peptides are so common across proteins. Presenting

them would lead to many non-specific matches when presented to CD4+ cells. Low-specificity peptides could lead to inappropriate activation because it would be difficult for the immune system to distinguish self from non-self.